



Hype: AI 趋势雷达

hype.replicate.dev 开源拆解 —— 一个只有 593 行代码的"AI 情报站"是怎么做出来的

Cloudflare Workers

D1 SQLite

Hono + Mustache

TypeScript

Apache-2.0 开源

仓库 github.com/replicate/hype | 线上版 hype.replicate.dev | 2026-10-03 学习产出 ·
F:\reminder\replicate_hype

⚡ 30 秒速览: 这个项目是什么

Replicate 官方开源的 **HN 风格 AI 趋势聚合站**: 每小时自动抓取 4 个平台 (GitHub / Replicate / HuggingFace / Reddit) 的火热 AI 项目, 用一套 **跨平台评分公式** 统一排名, 做成一个"Hacker News for ML"。推文说的"每天看 5 分钟, 比 90% 的人更早看见风口"就是它。

📦 体积: 9 个源码文件, 共 593 行

🌐 线上: hype.replicate.dev (254★)

📜 许可: Apache-2.0 (可自由部署/魔改)

💰 部署成本: Cloudflare 免费额度即可跑

🧪 本地实验: 6/6 全部 PASS

Q1 它是什么? 为什么"刷推特"不如看它?

Hype 是一个"趋势雷达": 推特的信息靠人转发(慢、带情绪、掺广告), 而它直接盯 **四个一线数据源的行动数据** —— 项目被标星、被调用、被下载、被讨论, 这些是**真金白银的行为信号**, 不是口头安利。

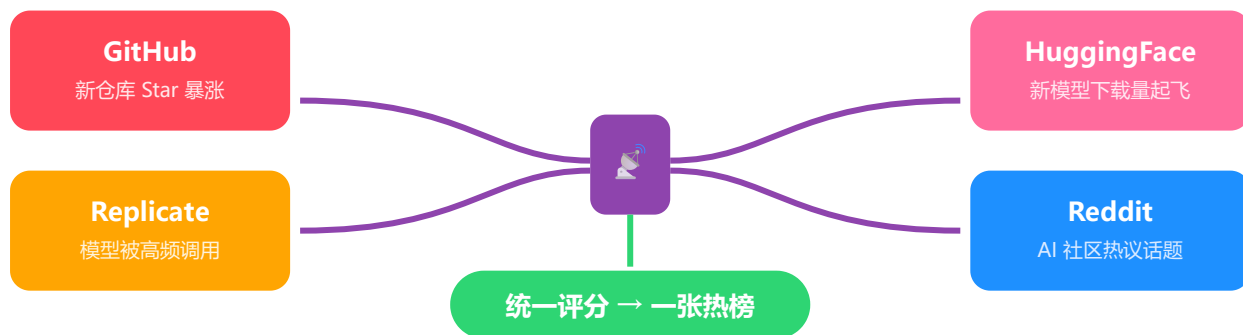


图1: 四个"行动数据"源汇入统一评分, 产出一张热榜

Q2 它凭什么知道"什么火"? 四个抓取器怎么工作?

核心文件 `src/fetchers.ts` (157 行) 里有 4 个抓取函数, 全部调各平台公开 API, 没有任何爬虫黑魔法:

抓取器	数据源 API	抓什么	热度原始值
fetchGitHubPosts	GitHub Search API	最近 7 天新建的 Python 仓库 (按 star 排序, 取前 500)	star 数
fetchReplicatePosts	Replicate API (要 token)	最近一周有更新的官方模型	run_count 调用次数
fetchHuggingFacePosts	HF 公开 API	最近改动的模型 (likes>1 且 downloads>1)	likes 数
fetchRedditPosts	Reddit JSON API	r/MachineLearning、r/LocalLLaMA、r/StableDiffusion 周榜	upvote 分数

为什么只抓 Python 新仓库? README 里作者吐槽: "现在 GitHub 上几乎每到一个新 Python 仓库都是 ML 仓库"——所以 Python + 新建 就是"新 AI 项目"的最好近似。4 个抓取器用 `Promise.all` 并行执行, 谁快走谁的, 互不拖累。

Q3 Star、下载量、upvote、调用次数——单位都不一样, 怎么一起排名?

这是全项目最精妙的一段, 只有 4 行——`scorePost` 评分归一化:

```
function scorePost(post: Post): number {
  if (post.source === "reddit") return post.stars * 0.3; // upvote 打 3 折
  if (post.source === "replicate") return Math.pow(post.stars, 0.6); // 调用次数开"压缩"
  return post.stars; // GitHub/HF star 原值
}
```

把四种单位不同的热度, 换算到同一张"分数卷子"上再排名。为什么不统一乘系数而是 $x^{0.6}$ 幂函数? 因为 `run_count` 动辄几万次, 直接比会碾压其他平台; 幂函数让大数"越大的部分增长越慢" ($32000^{0.6} \approx 504$, 和几千 star 同一量级)。Reddit 打 3 折则是因为 upvote 门槛低、水分数值虚高。



图2: `scorePost` 把四种单位压到同一量级 (实测输出, 见实验第 4 步)

Q4 垃圾项目怎么挡在门外？

热榜天然就是垃圾站攻击目标（蹭流量）。hype 用两道**确定性过滤器**（纯规则、不花一分钱 AI 费用）：

- ① **黑名单关键词**：仓库名/描述命中 nft / crypto / telegram / clicker / solana / stealer 直接拒收；
- ② **硬性条件**：username 为空拒收；HF 模型 likes≤1 或 downloads≤1 拒收；Replicate 模型 run_count≤1 拒收（没人用 = 不热）。

⚠️ 和已学的 **事实与判断分离** 同一思想：能不能上榜是**规则说了算的确定性判断**，全程没有 LLM 参与——0 成本、毫秒级、结果可复现。热度本身就是客观数字，根本不需要 AI 来“感觉”。

Q5 数据存哪？D1 是什么？

Cloudflare D1 = Cloudflare 出的**跑在边缘节点上的 Serverless SQLite**。你写的就是普通 SQL（建表迁移 migrations/0001_init.sql 就是一张 repositories 表），但不用装数据库、不用管备份扩容，按查询次数计费（免费额度 500 万次读/天）。

表结构极简：**id + source 联合主键**（同一项目可能同时出现在 GitHub 和 HF，靠 source 区分），外加 name/description/stars/url/created_at。写入用 **upsert** (INSERT ... ON CONFLICT DO UPDATE) ——**每小时重复抓到同一项目时只更新数值，不产生重复行**，天然幂等。

```
-- 一张表打天下 (migrations/0001_init.sql)
CREATE TABLE repositories (
  id TEXT NOT NULL, source TEXT NOT NULL,
  username TEXT, name TEXT, description TEXT,
  stars INTEGER DEFAULT 0, url TEXT,
  created_at TEXT, inserted_at TEXT DEFAULT (datetime('now')),
  PRIMARY KEY (id, source)      -- 联合主键防重复
);
```

Q6 网页怎么渲染出来的？

技术栈是 **Hono**（跑在 Workers 上的超轻 Web 框架，可类比 Express）+ **Mustache**（无逻辑模板引擎，模板里只有 {{变量}} 占位符，不允许写 if 循环之外的逻辑）。整个“渲染”就是：查 D1 → JS 里按 scorePost 重排 → 数据填进 127 行的 page.html 模板 → 吐出 HTML，带 5 分钟 CDN 缓存。

路由一共就 6 条：/（主页）、/api/posts（JSON 列表）、/api/last-updated、/api/update（触发抓取，GET/POST 各一份）、/docs（chanfana 自动生成的 OpenAPI 文档）。

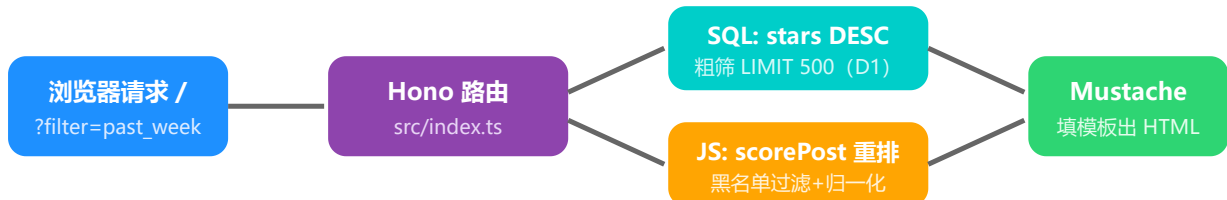


图3：一次请求的完整链路——两段式排序（SQL 粗筛 → JS 精排）

Q7 它部署在哪？Cloudflare Workers 是什么？

Cloudflare Workers = 把你的代码复制到全球 300+ 个机房的 Serverless 函数平台。没有服务器要管：不发请求时零计费，请求来了就近的节点执行。hype 整个后端就是一个 Worker（入口 src/index.ts 导出 fetch 处理函数 + 一个 scheduled 定时函数）。

对“怕运维”的个人项目来说，这是近乎零成本的完整后端：免费额度 10 万请求/天，静态站点 + API + 数据库（D1）+ 定时任务全家桶。这也是为什么它敢开源——你 fork 之后改个名字就能部署一个自己的版本。

Q8 数据多久更新一次？谁在背后定时跑？

答案有点反直觉：**不是 Workers 的 cron，而是 GitHub Actions**。仓库里的 .github/workflows/update_repos.yml 配了 cron: '0 * * * *'（每小时整点），Action 启动一个临时容器 → npm ci → 跑 scripts/updateContent.js → 用 API token 调线上 Worker 的 /api/update 端点 → Worker 执行 4 个 fetcher 并 upsert 入库。

为什么绕一圈？因为抓 5000 个 HF 模型的 README 很耗时，放在 Actions 的免费算力里跑，**Worker 只负责“接单”，脏活累活外包给 GitHub**——两边都在免费额度内。这是一个值得记住的模式：**定时任务不必和 Web 服务住在一起**。

Q9 怎么把它变成自己的？自部署要几步？

README 给的官方五步（需 Cloudflare 账号 + Replicate API token，token 只影响 Replicate 源，其余三源不需要）：

```
wrangler d1 create hype # ① 建数据库
# ② 把返回的 database_id 填进 wrangler.jsonc
wrangler d1 execute hype --remote --file=migrations/0001_init.sql # ③ 建表
wrangler secret put REPLICATE_API_TOKEN # ④ 存密钥
npm run deploy # ⑤ 上线全球
```

再把 GitHub Actions 的 workflow 里的目标 URL 指向你的 Worker，每小时自动更新就位。**全程零服务器、零月费。**

Q10 它开放了什么 API？怎么喂给自己的工具？

```
GET /api/posts?filter=past_week&sources=GitHub,Reddit
→ { "success": true, "result": [ {source, username, name, stars, url, ...}, ... ] }
```



```
GET /api/last-updated → 数据最后更新时间
GET /docs → 自动生成的 OpenAPI 交互文档
```

三个可玩的组合：① 把 JSON 接进自己的日报脚本/飞书机器人，每天早上自动推 5 条最热；② filter+sources 组合做“只看 Reddit 模型圈”的小雷达；③ 用 schema 校验（zod）保证的字段结构直接喂 LLM 做点评——榜单是确定性事实，LLM 只负责解读。

Q11 它有什么短板？（读懂局限才算真懂）

局限① 两段式排序会“误杀”。SQL 先按原始 stars 排序取 LIMIT 500，之后才做 scorePost 归一化。一个 Reddit 神帖（原始 upvote 30000+）如果别的平台垃圾条目太多，可能在 SQL 粗筛阶段就被挤出 500 名，根本活不到加分环节。正确做法是把归一化下推进 SQL——但对 hobby 项目，够用就好。

局限② 评分权重是拍脑袋的。0.3 和 0.6 没有任何校准依据，Reddit 权重调大 Reddit 条目立刻霸榜（我们实验里实测复现了这一点）。黑名单也只有 6 个词，换个马甲词就漏。

局限③ 只盯 Python 新仓库。Rust/Go 的 AI 基建项目（如很多推理引擎）完全不在雷达上——这是设计取舍，不是 bug。

学完它，你能干什么

- **每天 5 分钟情报扫描：**直接用线上版 hype.replicate.dev，或自部署一份加自己的过滤词；
- **做自己的“XX 雷达”：**照抄架构（fetcher×N + 归一化评分 + Worker + D1 + Actions 定时），换成你关注的领域（如量化圈、安全圈）；

- **接 API 做日报机器人**：/api/posts 的 JSON 喂给飞书/Telegram 推送，早上一条消息看全热点；
- **学会 Serverless 全栈最小件**：Hono 路由 + D1 SQL + Mustache 渲染 + Actions 定时，593 行全看懂，是入门 Cloudflare 全栈的最佳教材体量；
- **跨源评分归一化思维**：任何"把不同口径的数字放一起排名"的场景（电商好评/销量/复购）都能套用 scorePost 模式，并记得它"先截断后归一"的误杀教训。



主实验：不花一分钱、不申请任何 key，本地复现整个热榜

思路：**不调用真实 API**（免 token、免网络依赖），而是直接往本地 D1 造 9 条混合源数据，把 hype 当“排名引擎”来验证它的三大机制。环境：Windows 11 + Node v24.21.0，全部命令实测通过。

步骤 1 克隆并安装依赖

```
git clone https://github.com/replicate/hype repo
cd repo && npm install
# npm 11 若报 install-scripts 警告，执行：npm install-scripts approve workerd esbuild
npx wrangler --version # 验证：输出 4.56.0
```

✅ 预期结果：wrangler 4.56.0，node_modules 就绪。

步骤 2 初始化本地 D1 数据库

```
cp .dev.vars.example .dev.vars # token 留空也能跑本地实验
npx wrangler d1 execute hype --local --file=migrations/0001_init.sql
```

目的：在本地模拟器里建表。--local 表示一切发生在你电脑上，不碰云端。**--local 是 Serverless 开发的“模拟考场”**。

✅ 预期结果：输出 "success": true。

步骤 3 造 9 条测试数据 (exercise/seed.sql)

```
npx wrangler d1 execute hype --local --file=../exercise/seed.sql
```

数据分组：5 条评分对比组（GitHub 2000★ / GitHub 1000★ / Reddit 3000 赞 / HF 800 赞 / Replicate 32000 次调用）+ 2 条黑名单组（99999★ 的 crypto 垃圾 + 88888★ 的 nft 垃圾）+ 2 条时间窗组（9 天前的 old-repo、2 天前的 day-old）。

✅ 预期结果：连续 3 条 "success": true（DELETE + 2 段 INSERT）。

步骤 4 起本地服务器，验证三大机制

```
npx wrangler dev --port 8787 # Ready on http://127.0.0.1:8787
curl "http://127.0.0.1:8787/api/posts?filter=past_week"
```

验证① 跨源归一化——实测返回顺序（名字后为原始值）：

```
gh-a(2000) → gh-b(1000) → Reddit热帖(原始3000→900) → hf-model(800)
→ day-old(600) → repl-hot(原始32000→ $\wedge 0.6 \approx 504$ )
```

与 scorePost 公式预测逐位一致：Reddit 3000 赞被 3 折压到 1000★ 仓库之后；Replicate 32000 次调用被幂函数压到最后。

验证② 黑名单——返回里搜不到 moon-crypto-bot(99999★) 和 nft-minter(88888★)，star 再高也进不了榜。

验证③ 时间窗 + 源过滤：

```
curl "http://127.0.0.1:8787/api/posts?filter=past_day"
→ day-old 消失（它生于 2 天前，24h 窗口外）
curl "http://127.0.0.1:8787/api/posts?filter=past_week&sources=GitHub"
→ 只剩 gh-a / gh-b / day-old 三条
```

✅ 预期结果：排序预测全中、黑名单 0 泄漏、时间窗与源过滤正确。本步实测 **PASS**。浏览器打开 <http://127.0.0.1:8787> 还能看到和线上版一模一样的热榜页面。

步骤 5 改一个参数，亲眼看榜单洗牌

```
# 把 src/db.ts 里 reddit 的权重 0.3 改成 1.0（wrangler 热重载）
curl "http://127.0.0.1:8787/api/posts?filter=past_week"
→ Reddit热帖(3000) 跳到第 1 名！原来第 1 的 gh-a 被挤到第 2
git checkout -- src/db.ts # 看完恢复原样
```

目的：证明“排名完全由权重公式决定”。0.3 → 1.0，Reddit 原始值不再打折，3000 分立刻登顶——**你在实验里亲手复现了“权重即编辑立场”**：谁定权重，谁定什么是“热门”。

✅ 预期结果：Reddit 条目从第 3 位跳升第 1 位。本步实测 **PASS**。



实测踩到的坑（写给下一个学的人）

- ① **npm 11 拦截 postinstall**：装依赖时 workerd/esbuild 的安装脚本默认被拦，wrangler 会起不来——用 `npm install-scripts approve workerd esbuild` 批准后 rebuild；
- ② **.dev.vars 里的 token 留空不影响本地实验**：只要不走 `/api/update`，本地 D1 造数就能驱动整站，最适合无 key 学习；
- ③ **wrangler dev 热重载**：改 `src/` 下文件 6 秒内自动生效，做“改参数看变化”实验不用重启；
- ④ 首次起站会提示 types 可能过期（`wrangler types` 可修），不影响运行。

